

Programação Web I

Linguagem de Programação Java

Visão geral e revisão dos fundamentos



Roteiro

- Breve Histórico do Java
- Fundamentos da Linguagem Java
- Estrutura e Sintaxe Básicas
- Básico de Orientação a Objetos
- Exercícios
- Referências

Breve Histórico do Java

Como a linguagem Java surgiu?

- O **Green Project**:
 - Em 1991, a *Sun Microsystem* financiou uma pesquisa interna que tinha o objetivo de fazer com que dispositivos eletrônicos pudessem se comunicar entre si.
 - O chefe do projeto, **James Gosling**, criou uma linguagem de programação, chamada na época de **Oak** (carvalho).
 - O nome **Oak** já estava registrado como uma outra tecnologia, então a equipe sugeriu o nome de uma cafeteria local, **Java** (nome de uma ilha da Indonésia, origem do café).



Breve Histórico do Java

Como a linguagem Java surgiu?

- O **desenvolvimento lento** do mercado de dispositivos eletrônicos inteligentes - área hoje conhecida como Internet das Coisas (IoT), e a **rápida evolução da *World Wide Web*** fez com que a *Sun* visualizasse potencial da linguagem Java para a Web.
- Em 1995 a *Sun* anuncia o ambiente Java, sendo um **sucesso absoluto**.
- *Timeline* resumido do Java: <https://www.timetoast.com/timelines/historia-do-java>

Fundamentos da Linguagem Java

Revisão de fundamentos de linguagens de programação

- Antes de tratar especificamente do Java, vamos revisar um pouco sobre alguns conceitos fundamentais de **linguagens de programação**.
- **Código-fonte**:
 - Contém comandos da linguagem de programação em uso.
- **Código-objeto** e **Código de máquina**:
 - **Código-objeto** representa a versão traduzida de um arquivo de código-fonte para uma porção de código de máquina.
 - **Código de máquina** seria então o código binário (executável) final do programa para um processador específico.

Fundamentos da Linguagem Java

Revisão de fundamentos de linguagens de programação

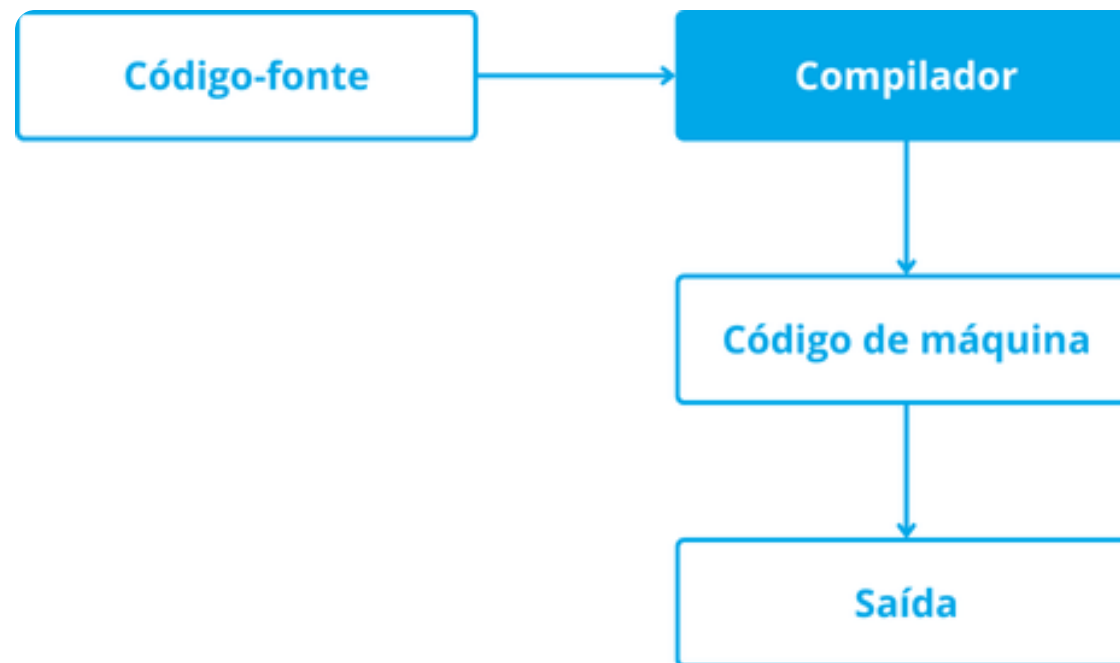
- **Compilador**

- Responsável pelo processo de **compilação**, que converte um código-fonte diretamente para um código de máquina que o processador pode executar, ou um código de mais baixo nível.
- Linguagens compiladas **tendem a ser mais rápidas e eficientes**.
- O processo de tradução é feito **todo de uma vez**. Só depois, o programa será executado (se não ocorrer erros no processo de compilação).
- **Verificação de Erros em Tempo de Compilação**. Ou seja, muitos erros podem ser detectados antes que o programa seja executado.

Fundamentos da Linguagem Java

Revisão de fundamentos de linguagens de programação

- **Compilador**



Fundamentos da Linguagem Java

Revisão de fundamentos de linguagens de programação

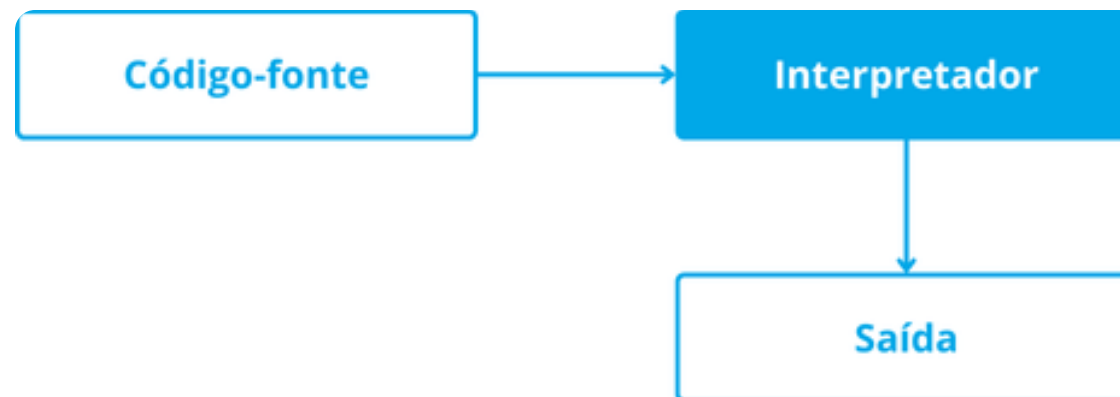
- **Interpretador**

- Responsável pelo processo de **interpretação**, que processa um comando por vez do código-fonte, traduzindo-o para uma representação de código de máquina correspondente.
- Ocorre **linha a linha**, e não traduzindo todo o código-fonte de uma vez.
- Em geral, possibilita maior **flexibilidade** e possibilita **depuração em tempo real**, a nível de código-fonte.
- **Detecção de Erros em Tempo de Execução**. Ou seja, alguns erros só são identificados quando a parte problemática do código é executada.
- **Desempenho**: Pode ser mais lento, pois a tradução acontece durante a execução do programa.

Fundamentos da Linguagem Java

Revisão de fundamentos de linguagens de programação

- **Interpretador**



Fundamentos da Linguagem Java

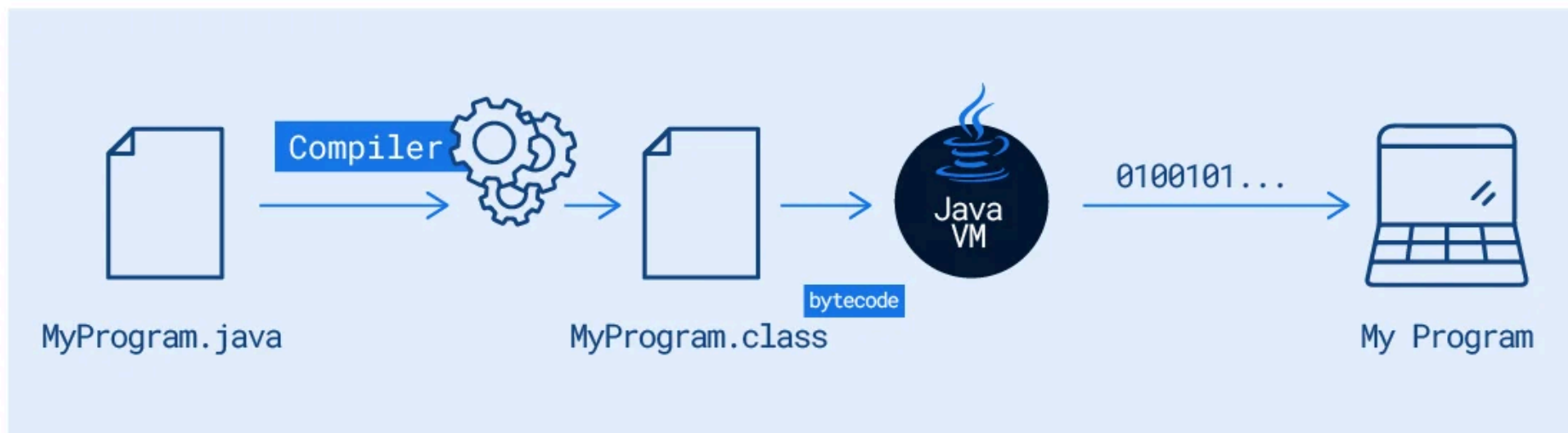
Processo de Tradução no Java

- **Modelo Híbrido de Tradução:**
 - O Java utiliza uma abordagem mista, onde compila o código-fonte em **bytecode**, que é então **interpretado** pela *Java Virtual Machine (JVM)*.
- **Java Virtual Machine (JVM):**
 - Uma **máquina virtual**, por conceito, simula uma máquina física.
 - A **máquina virtual Java** atua como uma **camada de abstração** entre a plataforma Java e o Hardware da máquina.
 - Dessa forma, um programa desenvolvido em Java é **independente** do Sistema Operacional. Isso, porque ele é compilado para **bytecode** que é então interpretado pela JVM.

Fundamentos da Linguagem Java

Processo de Tradução no Java

.java, .class, bytecodes



Fundamentos da Linguagem Java

Principais características do Java

- Entre os fatores que levaram à Linguagem Java ter tanto sucesso, podemos destacar que ela é:
 - **Orientada a Objetos:** suporte integral aos principais conceitos de orientação à objetos.
 - **Robusta:** altamente tipada, uso interno de ponteiros (não explícito ao programador), gerenciamento de memória automática pelo *Garbage Collector*, etc.
 - **Segura:** restringe acesso a arquivos, faz verificação dos *bytecodes*, vasta gama de APIs de criptografia e autenticação/autorização.
 - **Concorrente:** suporte a multithreads e aplicações concorrentes.
 - **Independência de plataforma:** os *bytecodes* podem ser interpretados para qualquer plataforma que tenha a JVM.

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Estrutura Básica** de um Programa Java:
 - **Package:** Define o pacote (ou namespace) que organiza as classes.
 - **Import:** Inclui bibliotecas necessárias para o código.
 - **Declaração de Classe:** Toda aplicação Java é baseada em classes.
 - **Método main:** Ponto de entrada da aplicação.

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Exemplo:**

```
package com.exemplo; // Declaração de pacote

import java.util.Scanner; // Import de classes

public class MeuPrograma { // Declaração da classe

    public static void main(String[] args) { // Método principal
        System.out.println("Olá, Mundo!"); // Saída no console
    }

}
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Tipos Primitivos** - Java possui 8 tipos primitivos:
 - `int` : números inteiros (ex.: 10)
 - `double` : números com ponto flutuante (ex.: 3.14)
 - `float` : números com ponto flutuante de menor precisão (ex.: 2.71f)
 - `boolean` : valores lógicos (true/false)
 - `char` : um único caractere (ex.: 'A')
 - `byte` , `short` , `long` : outros inteiros com diferentes intervalos.

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Declaração e inicialização**

- Variáveis devem ser declaradas com um tipo e podem ser inicializadas ao mesmo tempo.
- **Exemplo:**

```
int numero = 10;  
double valor = 3.14;  
boolean ativo = true;  
char letra = 'A';
```


Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- Estruturas de Decisão

- `if/else` - Usado para executar blocos de código com base em condições lógicas. **Exemplo:**

```
if (numero > 5) {  
    System.out.println("Número maior que 5");  
} else {  
    System.out.println("Número menor ou igual a 5");  
}
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- Estruturas de Decisão

- `switch` - Útil para testar o valor de uma variável contra diversos casos. [Exemplo:](#)

```
int opcao = 2;
switch (opcao) {
    case 1:
        System.out.println("Opção 1 selecionada");
        break;
    case 2:
        System.out.println("Opção 2 selecionada");
        break;
    default:
        System.out.println("Opção inválida");
        break;
}
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- Estruturas de Repetição

- **for** - Ideal para loops com um contador definido.

```
for (int i = 0; i < numero; i++) {  
    System.out.println("Iteração " + i);  
}
```

- **while** - Executa enquanto uma condição for verdadeira, verificando a condição antes de cada iteração.

```
int contador = 0;  
while (contador < numero) {  
    System.out.println("Contador: " + contador);  
    contador++;  
}
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- Estruturas de Repetição

- `do...while` - Garante que o bloco seja executado pelo menos uma vez, verificando a condição depois da execução.

```
int contador = 0;
do {
    System.out.println("Contador: " + contador);
    contador++;
} while (contador < numero);
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Método Main**

- Ponto de entrada padrão para qualquer aplicação Java.
- Assinatura padrão:

```
public static void main(String[] args) { ... }
```

- **Declaração de Métodos**

- Métodos podem receber parâmetros e retornar valores.

```
public int soma(int a, int b) {  
    return a + b;  
}
```

Estrutura e Sintaxe Básicas

Estrutura de Código e Sintaxe Básica da Linguagem

- **Declaração de Métodos**

- Métodos podem receber parâmetros e retornar valores.

```
public int soma(int a, int b) {  
    return a + b;  
}
```

- **Chamada de Métodos**

- Métodos podem ser chamados para realizar operações:

```
Exemplo obj = new Exemplo();  
int resultado = obj.soma(5, 3);
```

Estrutura e Sintaxe Básicas

Comentários e Convenções

- **Comentários**

- **Linha única:** Inicia com `//`
- **Bloco:** Envolvido por `/* ... */`
- **JavaDoc:** Inicia com `/** ... */` para documentar classes e métodos.

Estrutura e Sintaxe Básicas

Comentários e Convenções

- Comentários

```
// Este é um comentário de linha única
/*
 * Este é um comentário de
 * múltiplas linhas.
 */
/**
 * Método que realiza a soma de dois inteiros.
 * @param a primeiro número
 * @param b segundo número
 * @return a soma de a e b
 */
public int soma(int a, int b) {
    return a + b;
}
```


Estrutura e Sintaxe Básicas

Comentários e Convenções

- **Convenções de Nomenclatura:**

- **Classes:** Nome iniciando com letra maiúscula (CamelCase) – ex.: `CarrinhoDeCompras`
- **Métodos e Variáveis:** Nome em camelCase – ex.: `soma`, `numero`, `valorFinal`

- **Boas Práticas:**

- Escrever código limpo, legível e bem documentado.
- Manter o código modular e organizado para facilitar a manutenção e a escalabilidade.

Estrutura e Sintaxe Básicas

Entrada e Saída de Dados

- **Conceito:**
 - Entrada e saída (E/S) referem-se à forma como o programa interage com o usuário ou outros sistemas, permitindo ler dados de entrada (teclado, arquivos, etc.) e exibir resultados na saída (console, arquivos, etc.).
- **Principais Componentes:**
 - **Entrada:** `System.in`
 - **Saída:** `System.out` (para saída padrão) e `System.err` (para mensagens de erro)

Estrutura e Sintaxe Básicas

Entrada e Saída de Dados

- **Uso da Classe Scanner:**

- A classe `Scanner` (em `java.util`) é a forma mais simples e comum para ler dados do teclado. [Exemplo de Código:](#)

```
import java.util.Scanner;

public class EntradaExemplo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Digite seu nome: ");
        String nome = scanner.nextLine();
        System.out.print("Digite sua idade: ");
        int idade = scanner.nextInt();
        System.out.println("Olá, " + nome + "! Você tem " + idade + " anos.");
        scanner.close();
    }
}
```

Estrutura e Sintaxe Básicas

Entrada e Saída de Dados

- **Principais Métodos:**

- `System.out.print()` : Exibe o texto sem pular linha.
- `System.out.println()` : Exibe o texto e pula para a próxima linha.
- `System.out.printf()` : Exibe texto formatado, similar ao `printf` em C.
 - Alguns padrões de conversão no `printf` :
 - `%d`: Inteiro decimal.
 - `%f`: Número de ponto flutuante (decimal).
 - `%e` / `%E`: Notação científica (por exemplo, 3.1415E+00).
 - `%s`: String.
 - `%c`: Caractere.
 - `%b`: Booleano.

Estrutura e Sintaxe Básicas

Exemplo Integrado - E/S

```
import java.util.Scanner;

public class ProgramaCompleto {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Digite um número: ");
        int num = scanner.nextInt();

        System.out.print("Digite outro número: ");
        int outroNum = scanner.nextInt();

        int soma = num + outroNum;
        System.out.printf("A soma de %d e %d é: %d\n", num, outroNum, soma);

        scanner.close();
    }
}
```

Básico de Orientação a Objetos

Revisão dos Fundamentos de OO em Java

- **Paradigma de Programação:**

- A Orientação a Objetos é um paradigma que modela o software com base em "objetos", representações de entidades do mundo real.

- **Conceitos Básicos:**

- **Objetos:** Instâncias de classes que possuem estado (atributos) e comportamento (métodos).
- **Classes:** Moldes (plantas) que definem a estrutura e o comportamento dos objetos.

Básico de Orientação a Objetos

Revisão dos Fundamentos de OO em Java

- **Características Fundamentais:**
 - **Abstração:** Foco nos aspectos essenciais, ocultando detalhes irrelevantes.
 - **Encapsulamento:** Agrupamento de dados e métodos, protegendo informações internas.
 - **Herança:** Permite a criação de hierarquias e reutilização de código ("é um" relacionamento).
 - **Polimorfismo:** Habilidade de tratar objetos de diferentes classes por meio de uma interface comum.

Básico de Orientação a Objetos

Revisão dos Fundamentos de OO em Java

- **Benefícios e Impactos de OO**

- **Modularidade:**

- Facilita a organização do código em módulos independentes, melhorando a manutenção e o reuso.

- **Facilidade de Manutenção:**

- Alterações internas em uma classe não afetam outras partes do sistema, desde que a interface pública permaneça inalterada.

- **Reusabilidade:**

- Componentes orientados a objetos podem ser facilmente reutilizados em diferentes contextos e projetos.

Básico de Orientação a Objetos

Revisão dos Fundamentos de OO em Java

- **Benefícios e Impactos de OO**

- **Flexibilidade e Escalabilidade:**

- A combinação de herança e polimorfismo permite a extensão do sistema sem grandes reestruturações.

- **Melhor Modelagem do Mundo Real:**

- Permite representar entidades e suas interações de forma intuitiva, aproximando o software dos processos do mundo real.

Básico de Orientação a Objetos

Classes e Objetos - Exemplo Prático

```
public class Carro {  
    String modelo;  
    int ano;  
  
    public void acelerar() {  
        System.out.println("O carro está acelerando.");  
    }  
}  
  
// Instanciando um objeto:  
Carro meuCarro = new Carro();  
meuCarro.modelo = "Honda Civic";  
meuCarro.ano = 2025;  
meuCarro.acelerar();
```

Básico de Orientação a Objetos

Encapsulamento

- **Conceito:**
 - Agrupa dados (atributos) e os métodos que os manipulam, protegendo os detalhes internos.
- **Objetivo:**
 - Controlar o acesso aos dados e manter a integridade do objeto.
- **Práticas em Java:**
 - Uso de **modificadores de acesso**: private, protected e public.
 - Implementação de métodos **getters e setters**.

Básico de Orientação a Objetos

Encapsulamento

- Código de Exemplo:

```
public class Pessoa {  
    private String nome; // Atributo encapsulado  
  
    // Getter e Setter  
    public String getNome() {  
        return nome;  
    }  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Básico de Orientação a Objetos

Herança

- **Conceito:**
 - Permite que uma classe (subclasse) herde atributos e métodos de outra (superclasse).
- **Objetivo:**
 - Reutilizar código e modelar hierarquias do mundo real (relacionamento "é um").

Básico de Orientação a Objetos

Herança

- Código de Exemplo:

```
public class Animal {  
    public void comer() {  
        System.out.println("Comendo...");  
    }  
}  
  
public class Cachorro extends Animal {  
    public void latir() {  
        System.out.println("Au Au");  
    }  
}
```

Básico de Orientação a Objetos

Polimorfismo

- **Conceito:**
 - Permite que objetos de diferentes classes sejam tratados de forma uniforme, utilizando a mesma interface ou superclasse.
- **Objetivo:**
 - Flexibilizar e dinamizar a chamada de métodos, possibilitando comportamentos distintos conforme o objeto.
- **Práticas em Java:**
 - **Sobrescrita de Métodos:** Substituir a implementação de um método herdado.
 - **Interfaces e Classes Abstratas:** Fornecem contratos para diferentes implementações.

Básico de Orientação a Objetos

Polimorfismo

- Código de Exemplo:

```
public class Animal {  
    public void fazerSom() {  
        System.out.println("Som genérico");  
    }  
}  
  
public class Gato extends Animal {  
    @Override  
    public void fazerSom() {  
        System.out.println("Miau");  
    }  
}  
  
Animal meuAnimal = new Gato();  
meuAnimal.fazerSom(); // Exibe "Miau"
```


Bora Praticar!

Exercícios

1. Verificação de Maioridade:

Escreva um programa que leia a idade de uma pessoa e exiba uma mensagem indicando se ela é maior de idade ($\text{idade} \geq 18$). Use uma estrutura de decisão (if/else) para implementar a lógica.

2. Classe Produto:

Crie uma classe chamada `Produto` com os atributos `nome` (String), `preco` (double) e `quantidade` (int). Implemente:

- a. Um método para calcular o valor total em estoque ($\text{preço} \times \text{quantidade}$).
- b. Um método para exibir todas as informações do produto.

Escreva um programa que crie um objeto dessa classe e mostre os resultados.

3. Cálculo do Fatorial:

Desenvolva um programa que solicite ao usuário um número inteiro positivo e calcule seu fatorial utilizando um laço de repetição (pode ser `for` ou `while`). Exiba o resultado no console.

Exercícios

4. Classe Aluno e Aprovação:

Crie uma classe `Aluno` com os atributos `nome` (String) e `nota` (double). Implemente:

- a. Um método que verifica se o aluno foi aprovado ($\text{nota} \geq 7$).
- b. Um método que retorne o conceito do aluno, por exemplo:
 - a. A ($\text{nota} \geq 9$),
 - b. B (nota entre 7 e 8.9),
 - c. C ($\text{nota} < 7$).

Escreva um programa para testar essa classe.

5. Maior e Menor Número:

Escreva um programa que leia uma sequência de números (você pode definir um número fixo de iterações ou ler até um valor de parada) e determine qual é o maior e o menor número informado. Utilize estruturas de repetição para iterar sobre os valores.

Exercícios

6. Simulação de Conta Bancária:

Desenvolva uma classe `ContaBancaria` com os atributos `numero` (String) e `saldo` (double). Implemente os métodos:

- a. `depositar(double valor)` : aumenta o saldo.
- b. `sacar(double valor)` : diminui o saldo somente se houver fundos suficientes; caso contrário, exiba uma mensagem de erro.
- c. `consultarSaldo()` : retorna o saldo atual.

Escreva um programa para simular uma série de operações em uma conta.

Exercícios

7. Menu com Switch:

Crie um programa que exiba um menu de opções utilizando a estrutura `switch`. As opções devem ser:

- a. 1 – Cadastrar
- b. 2 – Consultar
- c. 3 – Atualizar
- d. 4 – Excluir

Para cada opção, exiba uma mensagem indicando a ação escolhida. Caso o usuário digite um valor inválido, exiba uma mensagem de erro.

Exercícios

8. Classe Retângulo:

Desenvolva uma classe `Retangulo` com os atributos `largura` e `altura` (ambos do tipo `double`).

Implemente:

- a. Um método para calcular a área.
- b. Um método para calcular o perímetro.

Escreva um programa que crie um objeto dessa classe com valores fornecidos pelo usuário e exiba os resultados.

9. Contagem de Números Ímpares:

Escreva um programa que leia dois números inteiros (início e fim de um intervalo) e utilize um laço `while` para contar quantos números ímpares existem nesse intervalo. Exiba o total ao final.

Exercícios

10. Calculadora Básica com OO:

Crie uma classe `Calculadora` com métodos para realizar as quatro operações básicas:

- a. `somar(double a, double b)`
- b. `subtrair(double a, double b)`
- c. `multiplicar(double a, double b)`
- d. `dividir(double a, double b)` (verificando se o divisor é diferente de zero)

Escreva um programa que permita ao usuário escolher a operação desejada (por meio de um menu) e, em seguida, informe dois números para realizar o cálculo.

Referências e Materiais Complementares

- H. M. Deitel, P. J. Deitel. [Java: Como Programar](#), 10a. Edição. Pearson, 2017.

 Disponível na BVU